Build systems

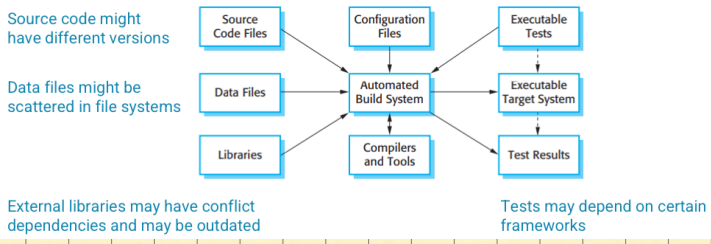
Fundamentally, all build systems have a straightforward purpose: transforming the source code into executable binaries

将源码转为可执行二进制

- Building is the process of creating a complete, executable software by compiling and linking the software components, external libraries, configuration files, etc.
- Building involves assembling a large amount of info about the software and its operating environment.

通过编译&链接各部分

For anything apart from very small systems, it always makes sense to use an automated build tool to create a system build



Dependency

Internal vs External Dependency

- **Internal dependency**: depend on another part of your codebase
- **External dependency**: depend on code or data owned by another team (either in your organization or a third party)

Task vs Artifact Dependency

- **Task dependency**: "I need to push the documentation before I mark a release as complete"
- **Artifact dependency**: "I need to have the latest version of the compute vision library before I could build my code"

任务

依赖特定版本

是否依赖自己代码

Dependency Scopes

- **Compile-time**: Foo uses classes or functions defined by `Bar`
- **Runtime**: Foo needs `Bar` (e.g., a database or network server) to be ready in order to execute
- **Test**: Foo needs `Bar` only for tests (e.g., JUnit)

Types of Build Systems

- Task-based Build Systems
- Artifact-based Build Systems

Task-based Build Systems

以task为单位

- In a task-based build system, the fundamental unit of work is the task
- Each task is a script of some sort that can execute any sort of logics, and tasks can specify other tasks as dependencies that must run before them
- Most major build systems (e.g., Ant, Maven, Gradle, Grunt, and Rake), are task based
- Most modern build systems require engineers to create buildfiles that describe how to perform the build (e.g., `pom.xml` for Maven)
- Engineers can write arbitrary code to execute any tasks during build
- Systems can't have enough information/control to always be able to run builds quickly and correctly

Drawbacks

难维护&查错

Difficulty maintaining & debugging build scripts

- Task-based build tools give too much power to engineers by allowing them to define any script as a task
- The scripts are easy places for bugs to hide and tend to grow in complexity
- The scripts end up being another thing that needs debugging & maintaining

Bug example 1:
Task A depends on task B to produce a particular file as output.
The owner of task B doesn't realize that other tasks rely on it, so they change it to produce output in a different location.
This can't be detected until someone tries to run task A and finds that it fails.

Bug example 2:
Task A depends on task B, which depends on task C, which is producing a particular file as output that's needed by task A.

The owner of task B decides that it doesn't need to depend on task C any more, which causes task A to fail.

Bug example 3:
The developer of a new task accidentally makes an assumption about the machine running the task, such as the location of a tool or the value of particular environment variables.
The task works on their machine, but fails whenever another developer tries it.

Bug example 4:
A task contains a nondeterministic component, such as downloading a file from the internet or adding a timestamp to a build.
Now, people will get potentially different results each time they run the build, meaning that engineers won't always be able to reproduce and fix one another's failures or failures that occur on an automated build system.

Bug example 5:
Tasks with multiple dependencies can create race conditions. If task A depends on both task B and task C, and task B and C both modify the same file, task A will get a different result depending on which one of tasks B and C finishes first.

Difficulty performing incremental builds

- Using a good build system, a small change doesn't require the entire code base to be rebuilt from scratch
- But for task-based build systems, it's hard to determine the change impact
- To ensure correctness, task-based build systems typically must rerun every task during each build

Difficulty of parallelizing build steps

- For the same reason (hard to determine task dependencies and change impact), task-based build systems are often unable to parallel task executions
- Huge wasting of development resources
- Hard to distribute the build across multiple machines

不适应增量模型
难并行 build

Artifact-based Build Systems

- In a build process, the role of system is **producing artifacts** (e.g., executable binary, documentation, etc.)
- Engineers still need to tell the system **what** to build, but **how** to do the build would be left to the system
- The approach that Google takes with `Blaze` (internal version) and `Bazel` (open-source version)

仅需告诉系统如何 build.

Buildfiles

BUILDFILES

- `Buildfiles` in artifact-based build systems like Blaze are a declarative manifest describing:
  - A set of artifacts to build
  - Their dependencies
  - Limited set of configurations
- Engineers run blaze by specifying a set of targets to build (the "what")
- Blaze is responsible for configuring, running, and scheduling the compilation steps (the "how")

```
java_binary(
  name = "MyBinary",
  srcs = ["MyBinary.java"],
  deps = [
    ":mylib",
  ],
)

java_library(
    name = "mylib",
    srcs = ["MyLibrary.java", "MyHelper.java"],
    visibility = ["//java/com/example/myproduct:__subpackages__"],
    deps = [
        "//java/com/example/common",
        "//java/com/example/myproduct/otherlib",
        "@com_google_common_guava_guava//jar",
    ],
)
```

Targets

TARGETS

- Buildfiles define targets; each target correspond to an artifact that can be created by the system
- java_binary: `binary` targets produce binaries that can be executed directly
- java_library: `library` targets produce libraries that can be used by binaries or other libraries

```
java_binary(
  name = "MyBinary",
  srcs = ["MyBinary.java"],
  deps = [
    ":mylib",
  ],
)

java_library(
    name = "mylib",
    srcs = ["MyLibrary.java", "MyHelper.java"],
    visibility = ["//java/com/example/myproduct:__subpackages__"],
    deps = [
        "//java/com/example/common",
        "//java/com/example/myproduct/otherlib",
        "@com_google_common_guava_guava//jar",
    ],
)
```

- Every target has
  - `name`, which could be used to reference this target
  - `srcs`, which define the source files that must be compiled to create the artifact for the target
  - `deps`, which define other targets that must be built before this target and linked into it

名称
源码
构建依赖

First Time Build

1. Parse every BUILD file in the workspace to create a graph of dependencies among artifacts.
2. Use the graph to determine the transitive dependencies of `MyBinary`; that is, every target that `MyBinary` depends on and every target that those targets depend on, recursively.
3. Build (or download for external dependencies) each of those dependencies, in order.
   - Bazel starts by building each target that has no other dependencies and keeps track of which dependencies still need to be built for each target.
   - When all of a target's dependencies are built, Bazel starts building that target. This process continues until every one of `MyBinary`'s transitive dependencies have been built.
4. Build `MyBinary` to produce a final executable binary that links in all dependencies that were built in step 3.

```
java_binary(
  name = "MyBinary",
  srcs = ["MyBinary.java"],
  deps = [
    ":mylib",
  ],
)

java_library(
    name = "mylib",
    srcs = ["MyLibrary.java", "MyHelper.java"],
    visibility = ["//java/com/example/myproduct:__subpackages__"],
    deps = [
        "//java/com/example/common",
        "//java/com/example/myproduct/otherlib",
        "@com_google_common_guava_guava//jar",
    ],
)
```

Benefits

Parallelism

- As Bazel knows that each target will only produce a Java library, it knows that **all it has to do is run the Java compiler rather than an arbitrary user-defined script**, so it knows that it's safe to run step 3 in parallel.
- This can produce an **order of magnitude performance improvement** over building targets one at a time on a multicore machine, because the artifact-based approach leaves the build system in charge of its own execution strategy so that **it can make stronger guarantees about parallelism**.

Incremental Builds

- Bazel knows that each target is the result only of running a Java compiler, and it knows that the output from the Java compiler depends only on its inputs, so **as long as the inputs haven't changed, the output can be reused**.
- Because Bazel knows about the properties of the tools it runs at every step, it's able to **rebuild only the minimum set of artifacts** each time while guaranteeing that it won't produce stale builds.

```
java_binary(
  name = "MyBinary",
  srcs = ["MyBinary.java"],
  deps = [
    ":mylib",
  ],
)

java_library(
    name = "mylib",
    srcs = ["MyLibrary.java", "MyHelper.java"],
    visibility = ["//java/com/example/myproduct:__subpackages__"],
    deps = [
        "//java/com/example/common",
        "//java/com/example/myproduct/otherlib",
        "@com_google_common_guava_guava//jar",
    ],
)
```

If `MyBinary.java` changes, Bazel knows to rebuild MyBinary but reuse `mylib`

If a source file for `//java/com/example/common` changes, Bazel knows to rebuild that library, `mylib`, and `MyBinary`, but reuse `//java/com/example/myproduct/otherlib`.

若依赖有变化，则会链式重新编译更新

Comparison

Task-based Build Systems

- Engineers define a series of steps to execute
- Imperative approach: flexible and powerful, but hard to guarantee correctness and parallelize

Artifact-based Build Systems

- Engineers declare a manifest describing the input (e.g., source files and tools like compilers) and output (e.g., binaries)
- Let the system figure out how to execute the build
- Declarative approach: strong guarantees about the correctness and easy to parallelize

Internal Dependency

A target depends on other targets in the same source repository

同一源码中一个目标依赖另一目标

Suppose target A depends on target B, which depends on a common library target C. Should target A be able to use classes defined in target C?

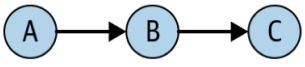


Figure 18-5. Transitive dependencies

Analysis: Suppose that B was refactored such that it no longer needed to depend on C. If B's dependency on C was then removed, A and any other target that used C via a dependency on B would break.

Solution: Google enforces **strict transitive dependencies** on Java code by default.

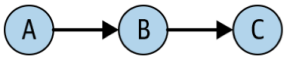


Figure 18-5. Transitive dependencies

- Blaze detects whether a target tries to reference a symbol without depending on it directly
- If so, the build fails with an error and a shell command that can be used to automatically insert the dependency
- Google also developed tools that automatically detect many missing dependencies and add them to a BUILD files without any developer intervention.

会找到非直接的依赖并强制引入.

External Dependency

A target depends on artifacts built and stored outside of the project and typically accessed via Internet

依赖第三方库

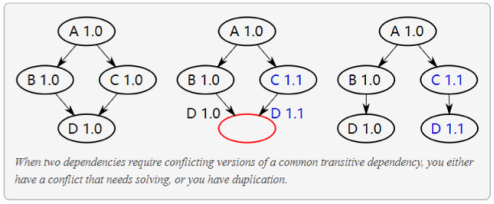
Reliability Risks
If the third-party source (e.g., a maven repository) goes down, your entire build might grind to a halt if it's unable to download an external dependency.

How to address
The problem can be mitigated by **mirroring** any artifacts you depend on onto servers you control and blocking your build system from directly accessing third-party artifact repositories like Maven Central.

Where are dependencies hosted?

- Maven Central (https://repo.maven.apache.org/maven2/)
- Ubuntu Packages (https://packages.ubuntu.com/)
- Python Package Index (https://pypi.org/)
- NPM Public Registry (https://registry.npmjs.org/)

Diamond Dependency Issues lead to conflicts and unexpected results



https://www.tedinski.com/2018/03/27/maven-design-case-study.html

Semantic Versioning

SemVer is the widely used convention for versioning releases using a three decimal-separated integers

Format: {MAJOR}.{MINOR}.{PATCH} (e.g., 2.4.72 or 1.1.4.)

- MAJOR version change indicates a change to an existing API that can break existing usage
- MINOR version change indicates purely added functionality that should not break existing usage
- PATCH version change indicates non-API-impacting implementation details (bug fixes) that are viewed as particularly low risk

Additional labels for pre-release and build metadata are available as extensions (e.g., 1.0.0-beta, 3.1.0-alpha.1)

大版本号：API的变化可能改变了已有用法
小版本号：新增功能不更改现有API用法
补丁版本号：不影响API实现细节（通常是修bug）

Using SemVer, version requirement can be expressed as "anything newer than", excluding API-incompatible changes (i.e., major version changes)

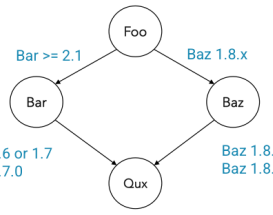
指定大于某个版本号时 MAJOR version 不应变化.
变的只有 MINOR、PATCH.

Declare dependency on "Bar ≥ 2.1"

√ • Bar 2.1
√ • Bar 2.1.0, 2.1.1
√ • Bar 2.2 onwards

× • Bar 2.0.x
× • Bar 3.x (some APIs in Bar were change incompatibly)

Using SemVer, we can model the dependency problem as: given a set of constraints (version requirements on dependency edges), can we find a set of versions for the nodes that satisfies all constraints?



SAT Solver:
Use Bar 2.1.0, Baz 1.8.1 and Qux 1.6.{latest}

Bar 2.1.0 depends on Qux 1.6 or 1.7
Bar 2.1.1 depends on Qux 1.7.0

Baz 1.8.0 depends on Qux 1.5.x
Baz 1.8.1 depends on Qux 1.6.x

取最新的.